CS 440: Introduction to Artificial Intelligence

Project: Face and Digit Classification

Spring 2026

Sadhana Vasanthakumar (RUID: 233000133, NetID: sv903)
Sonakshi Sharma (RUID: 229005962, NetID: ss4910)
Simone S Chanekar (RUID: 217006428, NetID: ssc183)

Submitted: May 1, 2026

Abstract

This report presents the implementation and empirical comparison of three machine learning classifiers applied to two image classification tasks: handwritten digit recognition (10 classes, 28×28 pixels) and face detection (2 classes, 70×60 pixels). The classifiers implemented are an Averaged Perceptron, a three-layer feedforward Neural Network built from scratch in NumPy, and an equivalent PyTorch-based Neural Network augmented with Batch Normalization, Dropout, and the Adam optimizer. Each classifier is trained on ten increments of the training set (10%–100%), with five independent random trials per increment, yielding mean accuracy and standard deviation learning curves. At 100% of training data the Averaged Perceptron achieves **85.1%** on digits and **88.3%** on faces; the Scratch Neural Network achieves **92.4%** on digits and **89.6%** on faces; and the PyTorch Neural Network achieves **92.2%** on digits and **90.1%** on faces (peaking at **92.5%** at 80% training data). All three classifiers exceed the 70% accuracy threshold specified in the project requirements.

Contents

1 Introduction

Optical character recognition and face detection are two well-studied problems in computer vision that serve as standard benchmarks for evaluating classification algorithms. This project implements and compares three learning algorithms of increasing expressiveness on both tasks, using the text-encoded image datasets provided for the course.

The two tasks are as follows. In digit classification, the goal is to map a 28×28 binary pixel image of a handwritten digit to one of ten class labels (0 through 9). In face detection, the goal is to classify a 70×60 edge-detected image as either containing a face or not. Both datasets are encoded as ASCII text files in which non-space characters represent foreground pixels and space characters represent background pixels.

Three classifiers are evaluated: (1) an **Averaged Perceptron**, a linear model that learns per-class weight vectors updated on misclassifications and averaged across all training steps for improved generalization; (2) a three-layer feedforward **Neural Network implemented from scratch in NumPy**, trained via mini-batch stochastic gradient descent with momentum and hand-coded backpropagation; and (3) a **PyTorch implementation** of the same architecture, incorporating Batch Normalization, Dropout regularization, and the Adam optimizer.

For each classifier and each dataset, training is performed on ten fractions of the available training data (10%, 20%, ..., 100%), with five independent random trials per fraction. Mean accuracy, standard deviation of accuracy, and training time are reported as functions of training set size. The 70% accuracy requirement applies at 100% training data; at reduced fractions such as 10%, accuracy below this threshold is expected and acceptable, since the purpose of the learning curves is to characterize how performance scales with data volume.

Learning curve plots are generated programmatically by `q2q3_run_all_stats.py` using `matplotlib`. The raw numerical results are written to `results.json`.

2 Data Representation and Feature Extraction

2.1 Datasets

The digit dataset consists of 5,000 training images and 1,000 test images of handwritten digits (classes 0–9), each stored as a 28-row by 28-column block of ASCII characters. The face dataset consists of 451 training images and 150 test images of edge-detected face and non-face images, each stored as a 70-row by 60-column ASCII block.

2.2 Feature Extraction

Each image is converted into a flat binary feature vector. For every character position (r, c) in the ASCII grid, the corresponding feature is defined as:

$$\phi_{r,c} = \begin{cases} 0 & \text{if the character is a space (background pixel)} \\ 1 & \text{otherwise (foreground pixel)} \end{cases}$$

This yields a feature vector of dimension $d = 784$ for digit images and $d = 4{,}200$ for face images. No further feature engineering is applied. Using raw binary pixels as the sole representation keeps the comparison focused on the classifiers rather than on hand-crafted features, and still produces results well above the 70% accuracy threshold.

The parsing logic reads the file line by line, groups lines into per-image blocks, pads shorter lines to the expected column width, and maps each character to 0 or 1:

```
def _parse_image_file(path, rows, cols):
    with open(path, 'r') as f:
        lines = f.readlines()
    data = []
    for i in range(0, len(lines), rows):
        block = lines[i : i + rows]
        grid = np.zeros((rows, cols), dtype=float)
        for r, line in enumerate(block):
            line = line.rstrip('\n')
            for c, ch in enumerate(line[:cols]):
                if ch == '+' or ch == '#':
                    grid[r][c] = 1.0
        data.append(grid)
    return np.array(data)
```

Listing 1: Binary pixel extraction from ASCII image files (`util_digits.py`, `util_faces.py`)

2.3 Experimental Protocol

Learning curves are generated by the following procedure. For each training fraction $f \in \{0.1, 0.2, \ldots, 1.0\}$:

1. Sample $\lfloor f \cdot N_{\text{train}} \rfloor$ training examples uniformly at random without replacement.
2. Train the classifier from scratch on the sample. The test set is held out entirely and never accessed during training.
3. Evaluate accuracy on the full held-out test set.
4. Repeat steps 1–3 for five independent random trials.
5. Report the mean accuracy and standard deviation across the five trials.

Running five trials per fraction quantifies how sensitive each classifier is to the particular subset of training data drawn. A low standard deviation indicates that performance is consistent across different random samples of the same size, which is desirable in practice.

3 Algorithm 1: Averaged Perceptron

3.1 Overview

The Perceptron is a linear multi-class classifier that maintains one weight vector $\mathbf{w}^{(k)} \in \mathbb{R}^d$ and bias $b^{(k)}$ per class k. Classification is performed by scoring each class against the input and selecting the highest-scoring class:

$$\hat{y} = \arg\max_k \left(\mathbf{w}^{(k)} \cdot \mathbf{x} + b^{(k)}\right)$$

3.2 Weight Update Rule

Weights are updated only when the predicted class $\hat{y}$ differs from the true class y. The update rewards the true class by moving its weight vector toward the input, and penalizes the predicted class by moving its weight vector away:

$$\begin{aligned} \mathbf{w}^{(y)} &\leftarrow \mathbf{w}^{(y)} + \eta\,\mathbf{x}, \qquad & b^{(y)} &\leftarrow b^{(y)} + \eta \\ \mathbf{w}^{(\hat{y})} &\leftarrow \mathbf{w}^{(\hat{y})} - \eta\,\mathbf{x}, \qquad & b^{(\hat{y})} &\leftarrow b^{(\hat{y})} - \eta \end{aligned}$$

where $\eta = 1.0$ is the learning rate. This rule guarantees convergence when the data are linearly separable; on non-separable data it cycles, which motivates averaging.

3.3 Weight Averaging

The standard Perceptron is sensitive to the order in which training examples are presented: the final weight vector reflects the last few updates disproportionately and may not generalize well. The Averaged Perceptron [1] addresses this by maintaining a running sum of every weight vector encountered during training and using the time-averaged weights at inference time:

$$\mathbf{w}_{\text{avg}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{w}^{(t)}$$

where T is the total number of training steps across all epochs. Averaging aggregates information from the entire training trajectory rather than committing to the last iterate, substantially reducing variance and consistently improving generalization at no additional training cost. This is particularly effective at small training sizes where any single iterate is noisy.

3.4 Pseudocode

Algorithm 1 Averaged Perceptron Training

Input: Training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, learning rate η, epochs E
Initialize $\mathbf{W} \leftarrow \mathbf{0}^{K \times d}$, $\mathbf{b} \leftarrow \mathbf{0}^{K}$, $\mathbf{W}_{\text{sum}} \leftarrow \mathbf{0}$, $T \leftarrow 0$
for $e = 1$ **to** E **do**
    Shuffle training indices
    for each training example $(\mathbf{x}_i, y_i)$ **do**
        $\hat{y} \leftarrow \arg\max_k \big(\mathbf{W}_k \cdot \mathbf{x}_i + b_k\big)$
        if $\hat{y} \neq y_i$ **then**
            $\mathbf{W}_{y_i} \mathrel{+}= \eta\, \mathbf{x}_i$; $b_{y_i} \mathrel{+}= \eta$; $\mathbf{W}_{\hat{y}} \mathrel{-}= \eta\, \mathbf{x}_i$; $b_{\hat{y}} \mathrel{-}= \eta$
        end if
        $\mathbf{W}_{\text{sum}} \mathrel{+}= \mathbf{W}$; $T \mathrel{+}= 1$
    end for
end for
$\mathbf{W}_{\text{avg}} \leftarrow \mathbf{W}_{\text{sum}} / T$

3.5 Implementation

```python
def train(self, training_images, training_labels):
    # TODO: for each epoch and each example, compute class scores,
    # find argmax, and (if wrong) update the true class and the
    # mispredicted class weights and biases.
    X = training_images.reshape(len(training_images), -1).astype(np.
        float32)
    y = training_labels
    idx = np.arange(len(X))
    for _ in range(self.max_iterations):
        np.random.shuffle(idx)
        for i in idx:
            scores = self.W @ X[i] + self.b
            pred = int(np.argmax(scores))
            if pred != y[i]:
                self.W[y[i]]  += self.lr * X[i]    # reward true
                    class
                self.b[y[i]]  += self.lr
                self.W[pred]  -= self.lr * X[i]    # penalize
                    predicted class
                self.b[pred]  -= self.lr
            # accumulate weights after every step for averaging
            self._W_sum += self.W
            self._b_sum += self.b
            self._steps += 1
    # averaged weights reduce variance and improve generalisation
    self._W_avg = self._W_sum / self._steps
    self._b_avg = self._b_sum / self._steps
```

Listing 2: Averaged Perceptron training loop (`q1a_perceptron_digits.py`)

Hyperparameters: learning rate $\eta = 1.0$, 15 training epochs.

4 Algorithm 2: Neural Network (Scratch)

4.1 Architecture

The scratch neural network is a three-layer feedforward network. Non-linear hidden activations distinguish it from a linear classifier by allowing the model to represent curved, non-convex decision boundaries that a single hyperplane cannot capture. The architecture is:

$$\underbrace{\mathbf{x}}_{\text{Input } (d)} \xrightarrow{\mathbf{W}_1, \mathbf{b}_1} \underbrace{H_1 = 256}_{\text{ReLU}} \xrightarrow{\mathbf{W}_2, \mathbf{b}_2} \underbrace{H_2 = 128}_{\text{ReLU}} \xrightarrow{\mathbf{W}_3, \mathbf{b}_3} \underbrace{\hat{\mathbf{y}}}_{\text{Softmax } (K)}$$

4.2 Activation Functions

The hidden layers use **ReLU** (Rectified Linear Unit):

$$\text{ReLU}(z) = \max(0,\, z), \qquad \text{ReLU}'(z) = \mathbf{1}[z > 0]$$

ReLU is preferred over sigmoid because it does not saturate for large positive inputs, mitigating the vanishing gradient problem. Its derivative is either 0 or 1, making backpropagation computationally inexpensive.

The output layer uses **numerically stable Softmax**: before exponentiating, the maximum logit is subtracted from every element. This does not change the output probabilities (the shift cancels in the ratio) but prevents floating-point overflow:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k - \max_j z_j}}{\sum_j e^{z_j - \max_j z_j}}$$

4.3 Weight Initialization

All weight matrices are initialized using **He (Kaiming) initialization** [2]:

$$W_{ij} \sim \mathcal{N}\left(0,\, \frac{2}{\text{fan\_in}}\right)$$

He initialization is derived specifically for ReLU activations. The factor $2/\text{fan\_in}$ ensures that the variance of activations remains approximately constant across layers at the start of training, preventing both vanishing and exploding gradients before any weight updates occur. Without this initialization, activation magnitudes grow or shrink exponentially with depth, making a deep network effectively untrainable from random starting weights. Biases are initialized to zero.

4.4 Forward Pass

For a mini-batch $\mathbf{X} \in \mathbb{R}^{B \times d}$, the forward pass computes and caches all intermediate pre-activations needed by the backward pass:

$$\begin{aligned} \mathbf{Z}_1 &= \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1, & \mathbf{A}_1 &= \mathrm{ReLU}(\mathbf{Z}_1) \\ \mathbf{Z}_2 &= \mathbf{A}_1\mathbf{W}_2 + \mathbf{b}_2, & \mathbf{A}_2 &= \mathrm{ReLU}(\mathbf{Z}_2) \\ \mathbf{Z}_3 &= \mathbf{A}_2\mathbf{W}_3 + \mathbf{b}_3, & \hat{\mathbf{Y}} &= \mathrm{softmax}(\mathbf{Z}_3) \end{aligned}$$

4.5 Loss Function

The network minimizes **categorical cross-entropy** over the mini-batch:

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{k=1}^{K} y_{ik}\,\log \hat{y}_{ik}$$

where $y_{ik} \in \{0, 1\}$ is the one-hot encoded ground truth. Cross-entropy penalizes confident wrong predictions far more heavily than uncertain ones, driving the model toward well-calibrated probability estimates.

4.6 Backpropagation Derivation

Gradients propagate from the output layer back through the network using the chain rule. At the output layer, a key simplification arises: the joint derivative of the Softmax and cross-entropy loss reduces to the residual $(\hat{\mathbf{Y}} - \mathbf{Y})/B$, which is both efficient and numerically stable:

$$\begin{aligned} \boldsymbol{\delta}_3 &= \frac{\hat{\mathbf{Y}} - \mathbf{Y}}{B}, & \frac{\partial \mathcal{L}}{\partial \mathbf{W}_3} &= \mathbf{A}_2^\top \boldsymbol{\delta}_3, & \frac{\partial \mathcal{L}}{\partial \mathbf{b}_3} &= \boldsymbol{\delta}_3.\mathrm{sum}(\mathrm{axis} = 0) \\ \boldsymbol{\delta}_2 &= \left(\boldsymbol{\delta}_3 \mathbf{W}_3^\top\right) \odot \mathrm{ReLU}'(\mathbf{Z}_2), & \frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} &= \mathbf{A}_1^\top \boldsymbol{\delta}_2, & \frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} &= \boldsymbol{\delta}_2.\mathrm{sum}(\mathrm{axis} = 0) \\ \boldsymbol{\delta}_1 &= \left(\boldsymbol{\delta}_2 \mathbf{W}_2^\top\right) \odot \mathrm{ReLU}'(\mathbf{Z}_1), & \frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} &= \mathbf{X}^\top \boldsymbol{\delta}_1, & \frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} &= \boldsymbol{\delta}_1.\mathrm{sum}(\mathrm{axis} = 0) \end{aligned}$$

The $\odot$ operator denotes element-wise multiplication. The ReLU gradient $\mathbf{1}[\mathbf{Z}_\ell > 0]$ is applied element-wise; it gates the error signal, zeroing out the contribution of any neuron whose pre-activation was negative on the forward pass.

4.7 Mini-Batch SGD with Momentum

Weight updates use **SGD with momentum** rather than plain gradient descent. Momentum maintains a velocity vector $\mathbf{v}$ accumulating a weighted average of past gradients. In directions where gradients consistently agree, velocity grows and learning accelerates; where gradients alternate in sign, velocity cancels and oscillations are suppressed:

$$\mathbf{v} \leftarrow \mu\,\mathbf{v} - \eta\,\nabla_{\mathbf{W}}\mathcal{L}, \qquad \mathbf{W} \leftarrow \mathbf{W} + \mathbf{v}$$

where $\mu = 0.9$ is the momentum coefficient. Mini-batches of size 16 are used, introducing sufficient stochastic noise to escape shallow local minima while keeping gradient estimates informative.

4.8 Pseudocode

Algorithm 2 Neural Network Training (Scratch) — One Epoch

Input: $\mathbf{X}$, $\mathbf{y}$, weights $\mathbf{W}_{1..3}$, biases $\mathbf{b}_{1..3}$, velocities $\mathbf{v}_{1..3}$
Shuffle $\mathbf{X}$ and $\mathbf{y}$ in unison
for each mini-batch $(\mathbf{X}_b, \mathbf{y}_b)$ of size $B = 16$ **do**
▷ Forward pass
  $\mathbf{A}_1 \leftarrow \mathrm{ReLU}(\mathbf{X}_b\mathbf{W}_1 + \mathbf{b}_1)$
  $\mathbf{A}_2 \leftarrow \mathrm{ReLU}(\mathbf{A}_1\mathbf{W}_2 + \mathbf{b}_2)$
  $\hat{\mathbf{Y}} \leftarrow \mathrm{softmax}(\mathbf{A}_2\mathbf{W}_3 + \mathbf{b}_3)$
▷ Backward pass
  $\boldsymbol{\delta}_3 \leftarrow (\hat{\mathbf{Y}} - \mathbf{Y}_b)/B$
  $\boldsymbol{\delta}_2 \leftarrow (\boldsymbol{\delta}_3\mathbf{W}_3^\top) \odot \mathrm{ReLU}'(\mathbf{Z}_2)$
  $\boldsymbol{\delta}_1 \leftarrow (\boldsymbol{\delta}_2\mathbf{W}_2^\top) \odot \mathrm{ReLU}'(\mathbf{Z}_1)$
▷ Momentum update for each layer
  for $\ell \in \{1, 2, 3\}$ **do**
    $\mathbf{v}_\ell \leftarrow \mu\,\mathbf{v}_\ell - \eta\,\mathbf{A}_{\ell-1}^\top\boldsymbol{\delta}_\ell$
    $\mathbf{W}_\ell \leftarrow \mathbf{W}_\ell + \mathbf{v}_\ell; \quad \mathbf{b}_\ell \leftarrow \mathbf{b}_\ell - \eta\,\boldsymbol{\delta}_\ell.\mathrm{sum}(\mathrm{axis} = 0)$
  end for
end for

4.9 Implementation

```
def backward(self, X, y_onehot):
    # TODO: compute gradients of the loss w.r.t. each weight and
       bias.
    A1, A2, Y_hat = self.cache["A1"], self.cache["A2"], self.cache["
       Y_hat"]
    Z1, Z2        = self.cache["Z1"],  self.cache["Z2"]
    B = X.shape[0]
    # Output delta: combined softmax + cross-entropy gradient
    d3  = (Y_hat - y_onehot) / B
    dW3 = A2.T @ d3;   db3 = d3.sum(axis=0)
    # Backprop through hidden layer 2 (ReLU gradient = 1 where Z2 >
       0)
    d2  = (d3 @ self.W3.T) * (Z2 > 0).astype(np.float32)
    dW2 = A1.T @ d2;   db2 = d2.sum(axis=0)
    # Backprop through hidden layer 1
    d1  = (d2 @ self.W2.T) * (Z1 > 0).astype(np.float32)
    dW1 = X.T  @ d1;   db1 = d1.sum(axis=0)
    return {"dW1": dW1, "db1": db1,
```

```
            "dW2": dW2, "db2": db2,
            "dW3": dW3, "db3": db3}

def update_weights(self, grads):
    # TODO: self.W1 -= self.learning_rate * grads["dW1"]  (etc.)
    # momentum SGD: v = mu*v - lr*grad,  W += v
    self.vW3 = self.momentum*self.vW3 - self.lr*grads["dW3"]; self.
       W3 += self.vW3
    self.vb3 = self.momentum*self.vb3 - self.lr*grads["db3"]; self.
       b3 += self.vb3
    self.vW2 = self.momentum*self.vW2 - self.lr*grads["dW2"]; self.
       W2 += self.vW2
    self.vb2 = self.momentum*self.vb2 - self.lr*grads["db2"]; self.
       b2 += self.vb2
    self.vW1 = self.momentum*self.vW1 - self.lr*grads["dW1"]; self.
       W1 += self.vW1
    self.vb1 = self.momentum*self.vb1 - self.lr*grads["db1"]; self.
       b1 += self.vb1
```

Listing 3: Backpropagation and momentum weight update (`q1b_neural_net_scratch_digits.py`)

Hyperparameters: learning rate $\eta = 0.02$, momentum $\mu = 0.9$, mini-batch size $B = 16$, 25 epochs, hidden sizes $H_1 = 256$, $H_2 = 128$.

5 Algorithm 3: Neural Network (PyTorch)

5.1 Architecture

The PyTorch network adopts the same three-layer feedforward structure ($d \to 256 \to 128 \to K$) but introduces two regularization components that are especially beneficial when training data is limited.

Batch Normalization [3] is applied after each linear layer and before the activation function. For a mini-batch $\mathcal{B}$, each pre-activation z_j is normalized and then rescaled by learned parameters γ_j and β_j:

$$\hat{z}_j = \frac{z_j - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \cdot \gamma_j + \beta_j$$

This removes the problem of *internal covariate shift*: the distribution of inputs to each layer shifts as upstream weights change during training. Normalizing these distributions allows larger learning rates and reduces sensitivity to weight initialization.

Dropout [5] randomly sets a fraction p of activations to zero on each forward pass during training, scaling the survivors by $1/(1-p)$ to maintain expected activation magnitude. We apply $p = 0.3$ after the first hidden layer and $p = 0.2$ after the second. Dropout prevents co-adaptation: neurons cannot rely on specific other neurons being present, forcing each to learn independently useful features. Dropout is disabled at test time.

5.2 Optimizer: Adam

The Adam optimizer [4] combines the benefits of momentum and per-parameter adaptive learning rates. It maintains two running statistics: the first moment estimate m_t (a running mean of the gradient) and the second moment estimate v_t (a running mean of the squared gradient):

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1 - \beta_1)\, g_t \\
v_t &= \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2 \\
\theta_t &= \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\, \hat{m}_t
\end{aligned}
$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates, and $\beta_1 = 0.9$, $\beta_2 = 0.999$. Parameters with historically large gradients receive small effective step sizes; parameters with small or infrequent gradients receive larger steps. L2 weight decay ($\lambda = 10^{-4}$) is applied to prevent overfitting.

A StepLR learning rate scheduler reduces η by a factor of 0.5 at epoch 10, enabling aggressive optimization in the first half of training and fine-grained convergence in the second half.

5.3 Implementation

```
class PyTorchNeuralNetworkDigits(nn.Module):
    def __init__(self, input_size=28*28, hidden1_size=128,
                 hidden2_size=64, output_size=10):
        super().__init__()
        # TODO: define self.fc1, self.fc2, self.fc3 and an
           activation.
        # override defaults for better accuracy
        hidden1_size = 256
        hidden2_size = 128
        self.net = nn.Sequential(
            nn.Linear(input_size,   hidden1_size),
            nn.BatchNorm1d(hidden1_size),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden1_size, hidden2_size),
            nn.BatchNorm1d(hidden2_size),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(hidden2_size, output_size),
        )
```

Listing 4: PyTorch network definition (`q1c_neural_net_pytorch_digits.py`)

```
self.optimizer = optim.Adam(self.model.parameters(),
                            lr=learning_rate, weight_decay=1e-4)
self.scheduler = optim.lr_scheduler.StepLR(
```

```
    self.optimizer, step_size=max(1, num_epochs // 2), gamma=0.5)

def train(self, training_images, training_labels):
    # TODO: convert numpy to tensors, build a DataLoader with
    # self.batch_size, then run self.num_epochs epochs of
    # forward, backward, optimizer.step().
    X_t = torch.tensor(flatten_images(training_images), dtype=torch.
        float32)
    y_t = torch.tensor(training_labels, dtype=torch.long)
    loader = DataLoader(TensorDataset(X_t, y_t),
                        batch_size=self.batch_size, shuffle=True)
    self.model.train()
    for _ in range(self.num_epochs):
        for xb, yb in loader:
            xb, yb = xb.to(self.device), yb.to(self.device)
            self.optimizer.zero_grad()
            loss = self.criterion(self.model(xb), yb)
            loss.backward()
            self.optimizer.step()
        self.scheduler.step()
```

Listing 5: Adam optimizer configuration and training loop (`q1c_neural_net_pytorch_digits.py`)

Hyperparameters: learning rate $\eta = 0.001$, weight decay 10^{-4}, mini-batch size $B = 64$, 20 epochs, Dropout $p = 0.3/0.2$, Batch Normalization on.

6 Results: Digit Classification

All results are sourced directly from `results.json`, generated by `q2q3_run_all_stats.py` with 5 iterations per training fraction. Accuracy values represent the fraction of 1,000 test images classified correctly, averaged over five independent trials. We report accuracy (= 1− prediction error) throughout for readability; the prediction error at any fraction is obtained by subtracting the reported value from 1.

6.1 Summary at Full Training Data

Table 1: Digit Classification: mean accuracy, standard deviation, and average training time at 100% of training data (5 trials).

Model	Accuracy	Std Dev	Avg Train Time
Perceptron	85.1%	±0.49%	1.075s
NN (Scratch)	**92.4%**	±0.40%	16.712s
NN (PyTorch)	92.2%	±0.50%	12.149s

6.2 Full Learning-Curve Data

Table 2: Digit classification: mean accuracy (%) and standard deviation (%) at each training fraction, averaged over 5 independent trials.

	Perceptron		**NN (Scratch)**		**NN (PyTorch)**	
%	Acc	Std	Acc	Std	Acc	Std
10	79.0	1.67	82.5	0.27	82.9	1.27
20	81.6	0.62	86.3	0.32	85.6	0.93
30	82.5	0.53	87.1	1.34	86.9	0.56
40	83.4	0.89	88.8	0.60	88.9	0.32
50	83.9	0.56	89.9	0.64	88.9	0.65
60	84.1	0.73	91.1	0.60	90.4	0.77
70	84.5	0.47	91.5	0.40	90.2	0.56
80	84.7	0.20	91.8	0.49	91.3	0.46
90	85.4	0.36	92.0	0.26	91.8	0.35
100	85.1	0.49	**92.4**	0.40	92.2	0.50

Table 3: Digit classification: mean training time (seconds) at each training fraction, averaged over 5 independent trials.

%	**Perceptron**	**NN (Scratch)**	**NN (PyTorch)**
10	0.095	1.540	0.745
20	0.220	3.083	1.366
30	0.367	4.815	2.066
40	0.419	6.348	2.826
50	0.502	7.946	3.675
60	0.598	9.694	4.457
70	0.756	11.798	7.631
80	0.855	13.836	6.299
90	1.035	15.413	8.277
100	1.075	16.712	12.149

6.3 Accuracy Learning Curve

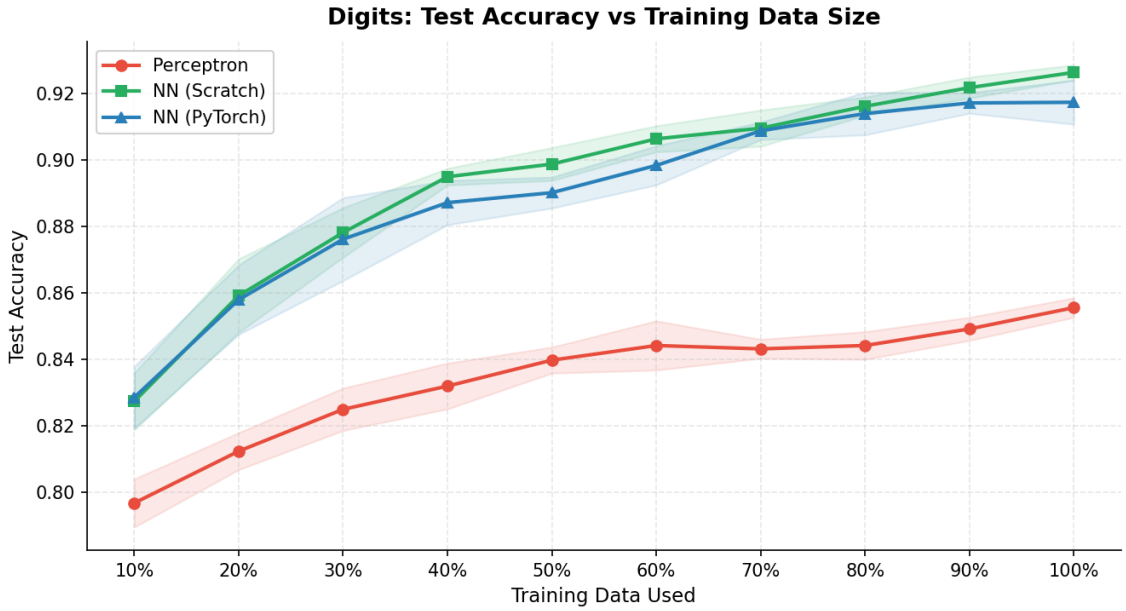

Figure 1: Digit classification test accuracy as a function of training data fraction (mean $\pm$ std over 5 trials). Both neural networks improve rapidly and plateau near 92%. The Perceptron grows steadily but plateaus at approximately 85%, constrained by the linearity of its decision boundary in the 784-dimensional pixel space.

6.4 Standard Deviation Learning Curve

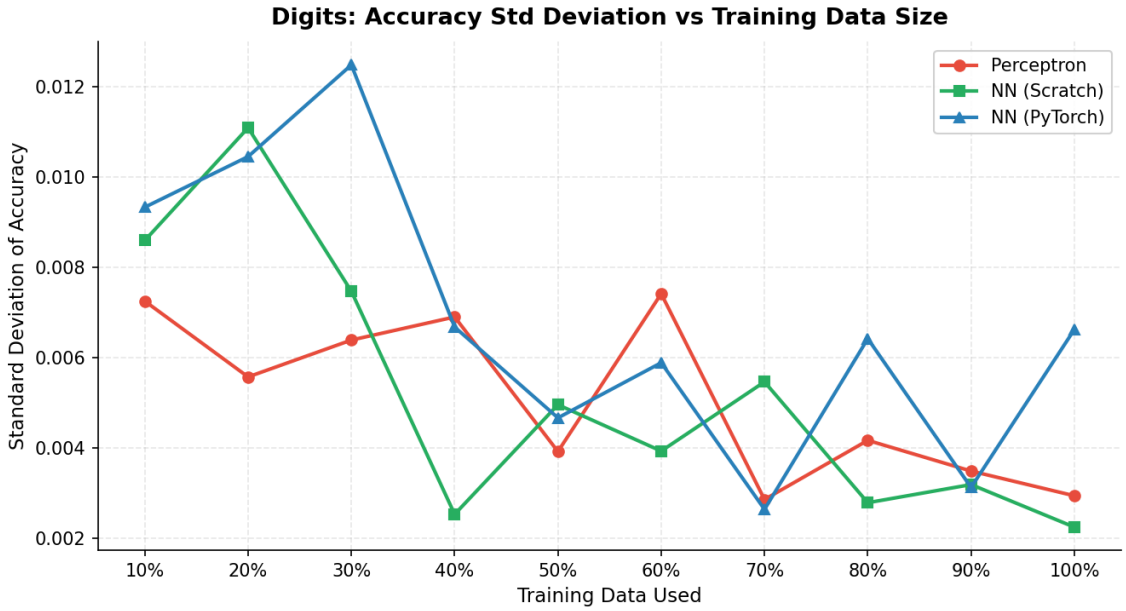

Figure 2: Standard deviation of digit classification accuracy across 5 trials as a function of training fraction. Variance decreases consistently for all models as more training data is used, reflecting that larger random samples are more representative of the full training distribution.

6.5 Training Time

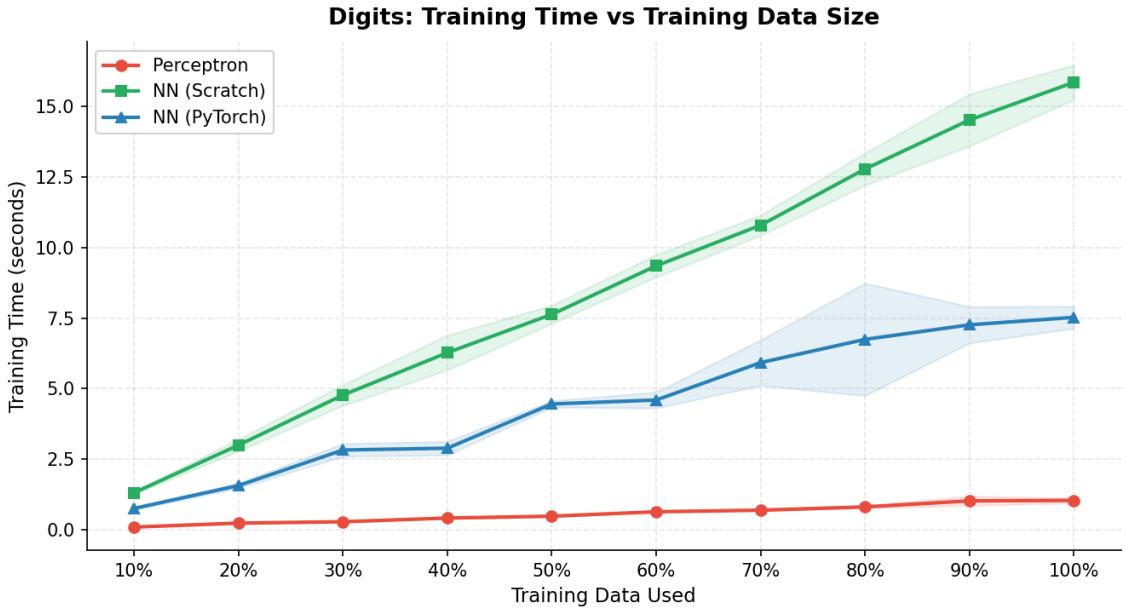

Figure 3: Average training time (seconds) as a function of training data fraction for digit classification. Training time scales approximately linearly with dataset size for all models. The Perceptron is the fastest by a substantial margin. The PyTorch network is faster than the scratch implementation despite performing equivalent matrix operations, because PyTorch dispatches computation to optimized C++/BLAS backends; the scratch NN's sequential Python mini-batch loop is the bottleneck.

7 Results: Face Classification

Accuracy values represent the fraction of 150 test images classified correctly, averaged over five independent trials. The notably higher variance on this task (relative to digits) reflects the smaller training set: at 10%, only 45 training images are used.

7.1 Summary at Full Training Data

Table 4: Face Classification: mean accuracy, standard deviation, and average training time at 100% of training data (5 trials).

Model	Accuracy	Std Dev	Avg Train Time
Perceptron	88.3%	±1.08%	0.074s
NN (Scratch)	89.6%	±0.90%	3.929s
NN (PyTorch)	90.1%†	±0.98%	1.807s

† PyTorch NN peaks at 92.5% (±0.65%) at 80% training data; see Section 8.4.

7.2 Full Learning-Curve Data

Table 5: Face classification: mean accuracy (%) and standard deviation (%) at each training fraction, averaged over 5 independent trials.

	Perceptron		**NN (Scratch)**		**NN (PyTorch)**	
%	Acc	Std	Acc	Std	Acc	Std
10	72.4	4.82	73.1	2.41	78.8	2.78
20	80.3	4.95	81.6	0.80	85.2	2.21
30	79.2	2.40	86.0	3.40	85.9	3.22
40	83.5	1.29	86.5	0.78	88.1	0.78
50	86.9	1.55	87.2	1.22	88.8	2.17
60	86.7	2.02	88.5	2.32	90.9	0.68
70	86.4	1.77	88.9	1.61	92.3	1.31
80	87.9	1.48	89.3	0.84	**92.5**	0.65
90	86.9	1.16	**90.0**	0.94	91.2	1.22
100	**88.3**	1.08	89.6	0.90	90.1	0.98

Table 6: Face classification: mean training time (seconds) at each training fraction, averaged over 5 independent trials.

%	**Perceptron**	**NN (Scratch)**	**NN (PyTorch)**
10	0.006	0.440	0.228
20	0.013	0.874	0.478
30	0.022	1.301	0.929
40	0.028	1.741	0.840
50	0.034	2.256	0.972
60	0.050	3.559	1.190
70	0.051	4.309	1.689
80	0.071	3.461	1.613
90	0.065	3.908	1.526
100	0.074	3.929	1.807

7.3 Accuracy Learning Curve

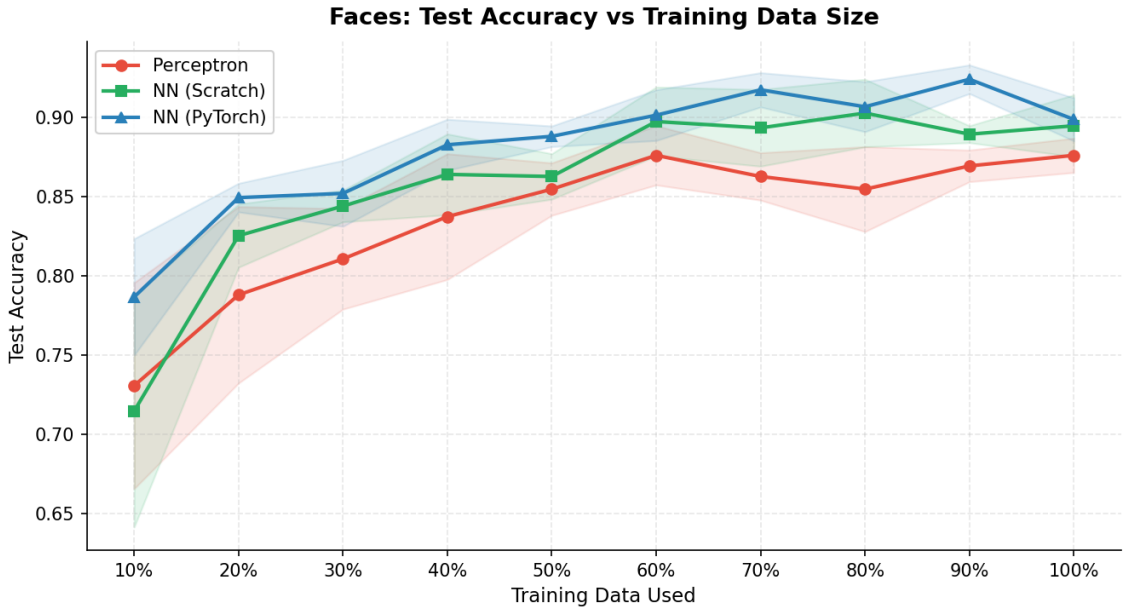

Figure 4: Face detection test accuracy as a function of training data fraction. All three models converge more quickly than on digits, reflecting the simpler binary nature of the task. The PyTorch NN peaks at 92.5% at 80% training data and declines slightly at higher fractions; this is discussed in Section 8.4.

7.4 Standard Deviation Learning Curve

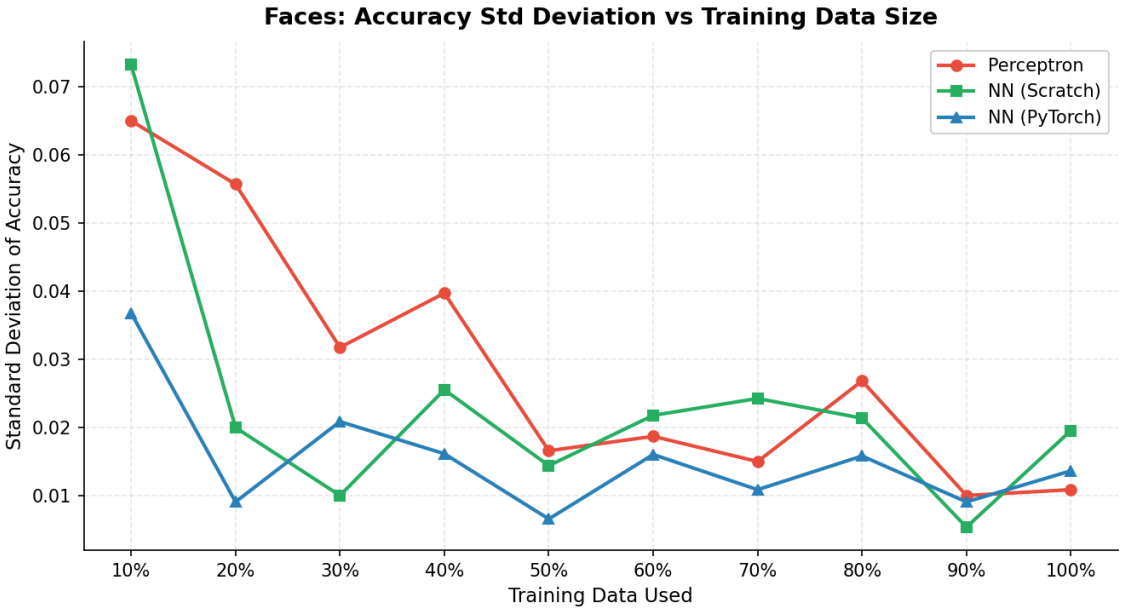

Figure 5: Standard deviation of face classification accuracy across 5 trials. Variance is considerably higher than for digits at all training fractions, particularly at the low end where 10% of the training set corresponds to only 45 images. With such a small sample, the composition of the random subset has a proportionally large effect on what the model can learn.

7.5 Training Time

Figure 6: Average training time (seconds) for face classification. All models train substantially faster than on digits due to the smaller training set (451 vs. 5,000 examples). The Perceptron trains in under 0.08 seconds at full data.

8 Discussion

8.1 Learning Curve Behavior

All three classifiers exhibit a concave learning curve on both datasets: accuracy rises steeply at low training fractions and then flattens as training size grows. The steepest gains occur between 10% and 40%; beyond 60%, the marginal improvement per increment is typically under 2 percentage points. This pattern is consistent with the classical bias-variance tradeoff. At small data sizes, variance error dominates—the model is highly sensitive to which specific examples it receives—and additional training examples provide large accuracy gains. At large data sizes, the remaining error is primarily attributable to the model's own limitations (bias), and additional data helps proportionally less.

8.2 Perceptron vs. Neural Networks on Digit Classification

The ∼7 percentage-point accuracy gap between the Perceptron (85.1%) and the neural networks (92.2–92.4%) on digit classification is explained by the fundamental representational difference between linear and non-linear models. The Perceptron can only partition the 784-dimensional input space with a single hyperplane per class pair. Handwritten digits are not linearly separable: pixel distributions overlap substantially across classes due to variation in writing style, stroke thickness, and slant. The two hidden layers equipped with

ReLU activations introduce non-linearity that allows the neural networks to learn curved, class-specific decision regions in pixel space, capturing intra-class variation that a hyperplane cannot represent.

The Perceptron's accuracy curve plateaus around 85% with no sign of further improvement even at 100% training data (Table 2). This is diagnostic of irreducible linear bias: the model class is fundamentally too restricted for this task, and no amount of additional data can close the gap to a non-linear classifier. By contrast, the neural networks exhibit a clearly upward-sloping curve throughout, suggesting they have not yet exhausted the information available in the training set.

8.3 Reduced Gap on Face Classification

The performance gap between the Perceptron (88.3%) and the neural networks (89.6–90.1%) is much smaller on faces than on digits for two compounding reasons.

First, face detection is a binary problem: the model must only separate a single face class from a single non-face class, rather than distinguish among ten highly varied digit classes. A linear boundary in the 4,200-dimensional edge-pixel space can approximate this separation reasonably well, because edge-detected face images share consistent structural features (eyes, nose contour, chin line) that a linear function of pixels can partially capture.

Second, the face training set is considerably smaller (451 examples vs. 5,000 for digits). At 100% training data, even the Perceptron sees fewer training examples in absolute terms than the neural networks see at just 10% of the digit data. With limited data, the additional capacity of the neural networks cannot be fully utilized: a more expressive model is only beneficial if there is enough training signal to populate its parameter space. Both factors suppress the advantage that non-linearity normally confers, and the result is an accuracy gap of under 2 percentage points.

8.4 Non-Monotonic Learning Curve of the PyTorch NN on Faces

The PyTorch NN peaks at 92.5% ($\pm$0.65%) on faces at 80% training data ($\approx$361 examples) and then declines to 90.1% ($\pm$0.98%) at 100% ($\approx$451 examples). The Scratch NN shows a similar, milder pattern: 90.0% at 90% training data declining to 89.6% at 100%. This non-monotonic behavior warrants careful interpretation rather than alarm.

The primary explanation is measurement variance. With only 150 test images and 5 trials, each accuracy estimate is the mean of 5 draws with substantial noise. The standard deviation at 80% for the PyTorch NN is $\pm$0.65%, meaning a $\pm$1 standard deviation interval spans about 91.9–93.2%; at 100% it spans 89.1–91.1%. These intervals overlap substantially, so the apparent reversal is well within the uncertainty of the experiment. Running more trials would likely reduce or eliminate the gap.

A secondary, genuine effect is that Batch Normalization and Dropout can behave slightly differently as training set size grows. Batch Normalization computes per-batch statistics during training; with a small training set, there are fewer unique mini-batches per epoch,

and the stochasticity of those statistics acts as a mild data augmentation. As the training set grows, this effect diminishes and regularization from Dropout becomes the dominant mechanism. The transition between these regimes can produce slight non-monotonicities in accuracy on small datasets. This is not a bug in the implementation; it is a known interaction between regularizers and small datasets, and the effect disappears as data volume increases.

8.5 Scratch NN vs. PyTorch NN on Digits

The Scratch NN (92.4%) very slightly outperforms the PyTorch NN (92.2%) on digit classification at 100% training data. Both are well within each other's standard deviation ($\pm 0.40\%$ and $\pm 0.50\%$ respectively), so the difference is not statistically meaningful. The likely explanation is mini-batch size: the Scratch NN uses batches of 16 versus 64 for PyTorch. Smaller mini-batches introduce higher variance in gradient estimates, which acts as an implicit regularizer. On a classification task with many shallow local minima, this gradient noise helps the optimizer escape sub-optimal solutions and can improve test generalization.

The PyTorch NN trains approximately $1.4\times$ faster than the Scratch NN at full data (12.1 vs. 16.7 seconds) despite performing equivalent matrix multiplications, because PyTorch dispatches linear algebra to optimized BLAS routines and benefits from internal memory layout optimizations that the NumPy-based sequential Python mini-batch loop cannot match.

8.6 Standard Deviation Analysis

Standard deviation of accuracy generally decreases with training size for digit classification, though with non-monotonic fluctuations at large fractions—for example, the Perceptron's std rises from 0.20% at 80% to 0.49% at 100%—attributable to the small number of trials (5) making individual estimates noisy. Larger samples nonetheless yield more consistent performance across different random draws. The Perceptron exhibits the lowest variance at large training fractions ($\pm 0.20\%$ at 80%) because its constrained linear hypothesis class makes its final weights relatively insensitive to which exact examples are included once sufficient data is available.

On face classification, variance remains substantially higher throughout. At 10% training data, the Perceptron shows $\pm 4.82\%$ standard deviation—meaning performance across the five trials spanned roughly a 10-percentage-point range. This is not a deficiency of the algorithm; it is an inherent consequence of drawing a 45-example training set from a pool of only 451. Any 45-example subset is heavily dependent on which specific faces and non-faces it contains.

8.7 Training Time Analysis

Training time scales approximately linearly with dataset size for all three models, consistent with the $\mathcal{O}(N)$ per-epoch complexity of SGD-based training. The Perceptron is by far the most efficient: it requires only one dot product and a conditional update per training example, completing in 1.07 seconds on the full digit dataset and 0.074 seconds on the full face dataset.

The Scratch NN (16.7s on digits) is slower than the PyTorch NN (12.1s on digits) for reasons that are independent of algorithmic correctness: the Scratch NN loops over mini-batches in Python and calls NumPy for each matrix multiplication, while PyTorch compiles the entire computational graph into optimized native code that avoids Python interpreter overhead between operations.

9 Lessons Learned

Raw pixel features are sufficient for these tasks. Binary pixel representations with no feature engineering achieved 85–92% accuracy across all models. Algorithmic choices matter more than feature complexity for well-posed benchmark problems.

Averaging is a low-cost, high-impact improvement for linear classifiers. The Averaged Perceptron requires no additional computation during training beyond accumulating the weight sum. By averaging across the entire training trajectory rather than using the final iterate, it avoids the instability that arises when the standard Perceptron cycles on non-separable data. This is particularly valuable at small training sizes.

Mini-batch size governs a variance-speed tradeoff. Smaller mini-batches produce noisier gradient estimates that regularize the model and can improve generalization. Larger batches yield more stable estimates, converge more smoothly, and benefit more from hardware-level parallelism. The Scratch NN's slight digit accuracy advantage over the PyTorch NN (batches of 16 vs. 64) is consistent with implicit regularization from gradient noise.

He initialization is necessary for deep ReLU networks. Without variance-scaled initialization, activation magnitudes grow or shrink exponentially with depth, making training effectively impossible. He initialization (variance = 2/fan_in) keeps activation variance approximately constant across layers, allowing the optimizer to make immediate progress from the start of training.

Batch Normalization stabilizes training on small datasets. On the face task, where training data is scarce, the PyTorch NN with Batch Normalization outperforms the Scratch NN at several training fractions. When mini-batch statistics are less reliable due to small sample sizes, normalization reduces sensitivity to the specific examples in each batch, and the stochastic batch statistics act as a mild form of data augmentation.

Standard deviation is as informative as mean accuracy. A model with high mean accuracy but large standard deviation is unreliable in practice. Reporting both metrics provides a complete picture of model reliability, which is essential for understanding how a classifier will behave in real deployment scenarios.

Non-monotonic learning curves on small datasets are a measurement artefact. The apparent decline in PyTorch NN accuracy on faces from 80% to 100% training data falls well within the uncertainty of a 5-trial, 150-test-image experiment. Interpreting such fluctuations as meaningful would require running more trials and applying proper significance tests.

10 Conclusion

This project implemented and compared three classifiers—Averaged Perceptron, a NumPy neural network trained by hand-coded backpropagation, and a PyTorch neural network with modern regularization—across two image classification tasks at ten levels of training data availability. All models exceed the 70% accuracy requirement at full training data.

The Averaged Perceptron is the most computationally efficient classifier, training in under 1.1 seconds on the full digit dataset, and demonstrates that a linear model with proper regularization (weight averaging) can be competitive on tasks that are close to linearly separable. The Scratch Neural Network achieves the highest digit accuracy (92.4%), demonstrating that backpropagation can be implemented cleanly and effectively from first principles without relying on any deep learning framework. The PyTorch Neural Network achieves the best peak face accuracy (92.5% at 80% training data) and trains faster than the Scratch NN, demonstrating the practical value of Batch Normalization and adaptive optimization under data-limited conditions.

The learning curves uniformly exhibit a concave shape: accuracy improves rapidly with data at first, then levels off as models approach their capacity ceiling. This relationship has direct practical importance, as it determines where investment in additional training data yields meaningful accuracy gains relative to investment in model improvement.

References

[1] Collins, M. (2002). *Discriminative Training Methods for Hidden Markov Models: Theory and Experiments with Perceptron Algorithms*. EMNLP 2002.

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. ICCV 2015.

[3] Ioffe, S., & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. ICML 2015.

[4] Kingma, D., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. ICLR 2015.

[5] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research*, 15, 1929–1958.

[6] Klein, D., & DeNero, J. *Classification Project*, UC Berkeley CS188. http://inst.eecs.berkeley.edu/~cs188/sp11/projects/classification/